

EVOLUTION AI: An End-to-End Bayesian Optimization and NURBS Pipeline for AI-Assisted Class-A Automotive Surface Design

Technical paper — platform version 1.2 (September 2026)

All system claims in this paper are anchored to the shipped source tree:

16 backend route modules with 118 HTTP endpoints (116 router endpoints plus 2 system endpoints), 11 ORM tables, 10 frontend pages, and a 489-case automated test baseline (200 algorithm-layer, 178 backend, 111 frontend).

Abstract

Automotive Class-A surface development sits at the intersection of industrial design, geometric modeling, and engineering validation. It traditionally requires weeks of iterative work between stylists, surfacing engineers, and CAE teams. We present **EVOLUTION AI**, an end-to-end web platform that integrates parametric automotive modeling, a NURBS-based Class-A surface pipeline with engineering-grade STEP export, a Gaussian process surrogate for data-efficient design optimization, and a fusion layer of deep-learning models and large language models (LLMs).

The system exposes four technical contributions. First, a **dual-pipeline geometry kernel** drives both a real-time triangle mesh for interactive preview and a from-parameters NURBS representation supporting G0/G1/G2 continuity analysis and a dependency-free, pure-Python STEP writer. Second, a self-contained **Bayesian optimization engine** — radial basis function Gaussian process solved with Cholesky factorization, coupled with Expected Improvement or Upper Confidence Bound acquisition — searches a 14-dimensional styling space and exports its observations as training data. Third, a **deep-learning/LLM fusion layer** combines a PyTorch vehicle-type network, CLIP-based six-axis semantic scoring of texture imagery, a unified proxy to seven commercial LLM providers with SSRF-safe fixed registries, and a local NURBS expert chat path. Fourth, an **end-to-end orchestration architecture** in which the web layer remains a thin, stateless shell over an independently installable algorithm

package, with graceful, truthful degradation (HTTP 503 or explicit fallback) whenever optional capabilities such as GPUs, Ollama, Redis, or CLIP weights are absent. We report the test baseline, optimizer convergence behavior, continuity measurements, and runtime performance, and discuss limitations and future work.

1. Introduction

1.1 Motivation

The exterior of an automobile is among the most demanding artifacts in industrial design. A production Class-A surface must simultaneously satisfy aesthetic intent (brand character, proportion, stance), geometric quality (continuity, curvature regularity, highlight behavior under reflection), and engineering constraints (manufacturability, assembly gaps, packaging). The conventional workflow is sequential and communication-heavy: concept sketches give way to tape drawings and digital surfaces; surfacing teams rebuild the design in NURBS; CAE and manufacturing feedback then trigger further rebuilds. Each loop may take days, and information is repeatedly translated between representations — sketch, mesh, NURBS, CAD — with inevitable loss and drift.

Three trends motivate rethinking this pipeline. (1) *Parametric, generative geometry* allows a vehicle to be defined by a compact vector of proportions and section parameters rather than by hand-built surfaces. (2) *Surrogate-based optimization*, particularly Bayesian optimization, promises data-efficient search when each geometry evaluation is expensive. (3) *Foundation models* — vision-language encoders and LLMs — introduce semantic understanding of style and natural-language access to domain expertise. The open problem is not any one of these capabilities, but their **integrated, trustworthy orchestration** in a single working system.

1.2 Goals and Design Principles

EVOLUTION AI was built under four principles:

1. **One design vector, every representation.** A single parameter set drives interactive mesh preview, NURBS surfaces, quality analysis, and engineering export, eliminating mesh-to-NURBS fitting error.
2. **Thin web shell, independent algorithm core.** The geometry and optimization logic live in an independently installable Python

package (`algorithm_model`); the FastAPI services only orchestrate.

3. **Data-efficient, inspectable optimization.** Bayesian optimization is implemented transparently in NumPy/SciPy so that every suggestion is reproducible, and all observations can be exported for downstream training.
4. **Truthful degradation.** Optional heavy dependencies (PyTorch, transformers/CLIP, Ollama, Redis) are genuinely optional. When they are missing the system returns explicit status codes or clearly labeled fallback data — it never fabricates a successful AI result.

1.3 Contributions

- A complete parametric vehicle pipeline in which a 22-field `CarParams` specification yields both a tessellated car (mesh path) and NURBS body/end surfaces with G1 blending, plus a pure-Python STEP writer requiring no OCCT-class dependency.
- A self-contained Bayesian optimization container for the 14-dimensional styling space, including session lifecycle APIs, warm-start from existing observations, dual acquisition functions, and an optimization-to-training data bridge.
- A heterogeneous AI fusion layer: a PyTorch classifier trained on a controlled synthetic-but-deterministic dataset, CLIP six-axis semantic scoring for texture-driven design, a seven-provider LLM proxy with encrypted key storage, and an on-premises NURBS expert chat.
- An end-to-end system evaluation on 489 automated tests, convergence experiments, and continuity/quality measurements.

1.4 Reproducibility Statement

Every quantitative figure in this paper corresponds to an executable test or script in the repository. The algorithm package ships a one-command self-check; the backend and frontend each carry their own automated test suites. The paper deliberately avoids reporting benchmark numbers that cannot be regenerated from the source tree.

2. Related Work

2.1 NURBS and Parametric Automotive Modeling

Non-uniform rational B-splines are the industrial standard for free-form shape representation because they unify conic sections and free-form

curves under one formulation and offer local control and strong continuity properties (Piegl & Tiller, 1997; Farin, 2002). Commercial surfacing systems rely on large kernels (e.g. Open CASCADE Technology) that carry substantial build and deployment weight. Our kernel instead implements the Cox–de Boor recursion, span finding (Piegl & Tiller Algorithm A2.1/A2.2), and NURBS curve/surface evaluation directly, with an optional Cython-accelerated path, and adds swept surfaces, fillets, and a STEP writer. This keeps server deployment lightweight while preserving an exact, parameter-driven representation.

Procedural vehicle modeling has been explored in graphics and in recent learning-based CAD work. Recent research such as NURBGen (AAAI 2026) investigates transformer-style generation of NURBS primitives, indicating strong interest in generative NURBS. Industrial research systems and shape-optimization platforms (e.g. topology and shape tools built on implicit or parametric representations) confirm the appetite for generative geometry, but they do not provide the full stylist-to-engineer web workflow. EVOLUTION AI is complementary: it uses an explicit, human-interpretable parameter vector rather than learned primitive distributions, which makes every generated car editable and auditable.

2.2 Bayesian Optimization

Bayesian optimization (BO) builds a probabilistic surrogate of an expensive black-box objective and uses an acquisition function to choose the next evaluation (Rasmussen & Williams, 2006). The classic Efficient Global Optimization framework uses a Gaussian process with Expected Improvement (Jones et al., 1998), and BO has become a workhorse for machine-learning hyperparameter tuning (Snoek et al., 2012). We adopt the same GP+acquisition backbone but target an *automotive styling* space with engineering units (millimetres and degrees), and embed the optimizer as a first-class service with persistent sessions and a bridge to the model training pipeline.

2.3 Deep Learning and Language Models in Design

Vision-language models such as CLIP (Radford et al., 2021) align image and text embeddings and can score imagery against textual style concepts. Parameter-efficient fine-tuning methods such as QLoRA (Dettmers et al., 2023) make domain adaptation of large language models practical on consumer GPUs, and LLMs increasingly serve as interfaces to specialized engineering knowledge. Design tools have begun to adopt these models

piecemeal. Our contribution is the *integration pattern*: CLIP semantics inform brand/parameter decisions, an LLM proxy unifies multiple commercial providers with strict server-side safety controls, and a local expert path keeps domain knowledge available without data leaving the premises.

3. System Overview

EVOLUTION AI is organized as five layers (Figure 1, conceptual).

L1	Frontend	Vue 3 + Vite + Three.js + Element Plus + Pinia 10 hash-routed pages, bilingual i18n, global auth guard
L2	Gateway	Vite dev proxy / Nginx; /api/v1 -> FastAPI:8000 public exposure via an HTTPS tunnel (cpolar)
L3	Backend	FastAPI + Pydantic v2; 16 thin route modules, 118 endpoints (116 module endpoints + 2 system endpoints); stateless orchestration over the algorithm package and the ML/LLM services
L4	Algorithm	algorithm_model (independently installable): car_modeling / freeform(NURBS) / surface_quality / storyboard
L5	Infrastructure	SQLite + file storage; optional Redis; background worker threads for long training tasks

Frontend (L1). The ten pages are: dashboard (/), parametric AI designer (/designer), project list with mock fallback (/projects), project detail (/projects/:id), deep-learning designer (/deep-learning), quality inspection (/quality), engineering delivery with import→modify→export sessions (/deliver), a guided demo (/demo), login (/login), and account (/account). Routing uses hash history for static-host compatibility, and a global guard redirects unauthenticated users away from protected views.

Backend (L3). Sixteen route modules cover projects, model generation, files, export, workflows, quality, deep-learning training, dashboard, optimization/delivery, brand/parameter queries, Bayesian optimization, textures, import-export, LLM proxy, and authentication with API-key management. Persistence uses eleven ORM tables: projects , model_files , workflows , workflow_steps , quality_reports , parameter_sets ,

`model_variants` , `users` , `api_keys` (Fernet-encrypted secret material),
`parameter_records` , and `training_tasks` .

Algorithm core (L4). The package is versioned and tested independently (200 pytest cases plus a five-module self-check). It exposes high-level entry points for whole-car generation, free-form surface construction, surface-quality evaluation, optimization, and storyboard rendering. Because web services call these entry points rather than embedding geometry logic, the core can be upgraded or replaced without touching the HTTP layer.

4. The NURBS-Based Class-A Surface Pipeline

4.1 Parameterization

A vehicle is described by a 22-field `CarParams` structure covering global proportions (length, width, height, wheelbase, track width, ground clearance), section and overhang data, angles (windshield, rear window, rear slant), wheel data, and cabin/roof proportions. Brand- and body-type-specific presets (sedan, SUV, sports, luxury) seed the structure; designers then perturb individual fields. Critically, *both* geometry pipelines consume the same object.

4.2 Dual Geometry Paths

Mesh path. A parameterized body is constructed from longitudinal and circumferential section rings (default discretization 48×24) and assembled with wheels, lights, grille, mirrors, glass and other parts via a component assembler. The assembler reports per-part vertex/face counts, bounds, and a stable part color so the front end can render and identify components. Meshes feed the Three.js viewport and STL/GLB export where tessellated output is sufficient.

NURBS path. A separate builder constructs the body as a genuine NURBS surface from a control grid, and dedicated modules build front and rear end caps with *G1 tangent blending* against the body. Supporting free-form operators include swept surfaces and fillet surfaces. Because surfaces are generated directly from parameters — not reverse-fitted from a triangulation — there is no fitting residual, and the designer's edits propagate to exact control-point changes.

4.3 Continuity and Surface Quality

Class-A assessment is performed at two granularities.

A *continuity checker* operates on named NURBS surfaces. It samples each of the four parameter-space boundaries per surface (50 samples per edge by default), estimates normals by finite differences of surface evaluation, and matches boundary points with a `cKDTree` nearest-neighbor search combined with a 4×4 edge-pair combinatorial match. Default thresholds are **0.1 mm for G0** positional continuity and **1.0° for G1** tangent continuity, with a looser 5° "visually smooth" band. The checker emits TXT/CSV reports and verifies design intent against expected gap and angle values.

A *surface-quality* package provides mesh-oriented curvature analysis — G0/G1/G2 level statistics from discrete curvature, highlight/reflection lines, and smoothing/optimization operators (including simulated annealing-based fairing). The two granularities are deliberately complementary: exact NURBS continuity at seams, and curvature-distribution analysis over the tessellated body for interactive feedback.

4.4 Engineering Export: STEP Without a Heavy Kernel

The free-form module contains a pure-Python STEP writer (`step_writer.py`) that emits ISO 10303-21 constructs for the NURBS geometry. Avoiding an Open-CASCADE-sized dependency matters for container size, cold-start time, and deployment in resource-constrained environments. The repository also ships example assemblies, standard-operating-procedure checklists, and FreeCAD verification macros so exported files can be opened and inspected in an independent kernel. IGES output and engineering format export are available alongside STL/GLB on the mesh side.

4.5 Pipeline Summary

The design-to-engineering chain is therefore:

```
CarParams → mesh assembly → interactive preview / STL / GLB
      ↳ NURBS body + G1 ends → G0/G1/G2 analysis → STEP/IGES
```

Every arrow is a pure function of the same parameter vector, which is what allows the optimizer in the next section to treat the entire styling and quality apparatus as a single evaluable function.

5. The Bayesian Optimization Engine

5.1 The Styling Search Space

The optimizer's default space contains 14 continuous dimensions aligned exactly with the training pipeline's feature order:

Parameter	Bounds (mm / °)
overall_length	4000 – 6200
overall_width	1750 – 2150
overall_height	1100 – 2050
wheel_base	2400 – 3700
track_width	1450 – 1900
ground_clearance	80 – 280
hood_length	700 – 1700
roof_height	300 – 1050
wheel_diameter	600 – 850
windshield_angle	18 – 50
rear_window_angle	10 – 50
rear_slant_angle	5 – 55
front_overhang	750 – 1150
rear_overhang	850 – 1350

The `ParameterSpace` class validates names (uniqueness), bounds (`max > min`), and values (unknown, missing, or out-of-range inputs raise explicit errors that the route layer maps to HTTP 400). All GP math runs in the unit cube after linear normalization; outputs are standardised to zero mean and unit variance before fitting.

5.2 Gaussian Process Surrogate

The surrogate is a Gaussian process with an isotropic squared-exponential (RBF) kernel:

$$k(\mathbf{x}, \mathbf{x}') = \exp\left(-\frac{\|\mathbf{x} - \mathbf{x}'\|^2}{2\ell^2}\right),$$

with default length scale ($\ell = 0.25$) in normalized space and jitter (10^{-6}) on the diagonal for numerical stability. Training assembles $(\mathbf{K} + \sigma^2 \mathbf{I})$, obtains the Cholesky factor $(\mathbf{L})$, and solves $(\boldsymbol{\alpha} = \mathbf{K}^{-1} \mathbf{y})$ through two triangular solves. Prediction at new points returns the posterior mean

$$[\mu(\mathbf{x}_*) = \mathbf{k}_*^{\top} \boldsymbol{\alpha}]$$

and standard deviation

$$[\begin{aligned} s(\mathbf{x}_*) &= \sqrt{\mathbf{k}_*^{\top} \mathbf{K}^{-1} \mathbf{k}_* - \mathbf{v}^{\top} \mathbf{v}}, \\ \mathbf{v} &= \mathbf{L}^{-1} \mathbf{k}_*, \end{aligned}]$$

clipped at a small positive floor. This is the textbook GP formulation (Rasmussen & Williams, 2006); implementing it directly keeps the engine free of any dependency beyond NumPy and SciPy (the latter supplies the normal CDF).

5.3 Acquisition Functions

Two acquisitions are supported. **Expected Improvement** over the current best standardised value (f^+), with exploration parameter ($\xi = 0.01$):

$$[\begin{aligned} \mathrm{EI}(\mathbf{x}) &= (\mu - f^+ - \xi) \Phi(z) + s \phi(z), \\ \text{quad } z &= \frac{\mu - f^+ - \xi}{s}, \end{aligned}]$$

where (ϕ, Φ) are the standard normal density and CDF. **Upper Confidence Bound**, with ($\kappa = 2.0$):

$$[\mathrm{UCB}(\mathbf{x}) = \mu + \kappa s.]$$

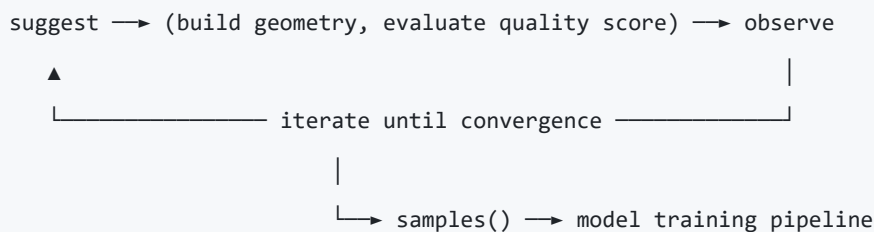
Minimization objectives are converted internally by negating scores, so both acquisitions and the best-selection logic have a single maximization implementation.

5.4 Suggestion Strategy

With fewer than two observations the surrogate is uninformative, so the engine draws random space-filling points. Once enough observations exist, it fits the GP, generates a candidate pool of 4096 random points in the unit cube, scores the entire pool with the acquisition function, and returns the top-(n) candidates denormalized to engineering units. A large random pool is a simple, derivative-free optimizer for the acquisition: it is embarrassingly parallel, has no local minima to tune, and makes multi-point suggestions (sorted by acquisition value) straightforward.

5.5 Session Lifecycle and the Training Bridge

Optimization state is organized into sessions stored in a thread-safe in-process registry. Each session carries its parameter space, goal direction, acquisition choice and hyperparameters, RNG seed, and the full observation list. The HTTP surface provides session creation/deletion, suggestion, observation ingestion (`observe`), best retrieval, and sample export. The export format uses feature dictionaries whose key order matches the training feature order, so an optimization campaign can be replayed as a supervised dataset:



This closes the loop between *search* (the optimizer proposes styling candidates) and *learning* (the accumulated observations become training data for downstream networks).

6. Deep Learning and LLM Fusion

6.1 The PyTorch Vehicle-Type Network

The training module builds a compact multilayer perceptron that maps the 14-dimensional normalized styling vector to vehicle type (six classes in the shipped configuration; the hidden layer has 48 units). Training runs in a background daemon thread so the HTTP API returns a task object

immediately; clients poll task status, logs, and progress, and may cancel.

Tasks persist in the `training_tasks` table.

The training dataset is *synthetic but deterministic*: centroids are taken from real per-class preset centers, parameter-specific noise scales govern sampling, and explicit style/brand transforms perturb designated dimensions. Determinism (seeded generators) makes training runs and tests reproducible, while the centroid/noise structure reflects real proportion ranges. A quality-metrics function derives aerodynamic and proportion indicators from the raw parameters. When PyTorch is absent, capability endpoints report the fact honestly and training endpoints fail cleanly rather than emulating results.

6.2 CLIP Semantic Scoring for Textures and Brand DNA

A dedicated semantic module wraps `openai/clip-vit-base-patch32` (loaded lazily, ~605 MB, with offline loading when weights are cached). It defines **six style axes**, each with a Chinese label and concrete English visual prompts — an important prompt-engineering choice because CLIP responds far better to specific visual descriptions than to abstract adjectives.

Given a texture or pattern image, the module encodes the image once and scores it against precomputed text embeddings to produce a value in $[0,1]$ per axis (0.5 neutral, above 0.5 leaning positive).

These semantic scores serve two consumers. The texture route fuses semantic features with geometric features when analyzing and applying parameterized textures (for example lamp/lens patterns). The brand knowledge layer maps the six-axis semantic vector onto brand profiles and recommended vehicles. A Rhino plugin can call the same API for a "pattern image → CLIP semantics → automatic parameter adjustment" workflow, surfacing availability status ("CLIP semantics available" vs. "geometric features only") rather than hiding it.

6.3 Unified, SSRF-Safe LLM Proxy

Natural-language capabilities are mediated by a proxy whose provider registry is fixed server-side. Seven providers are supported — Baidu ERNIE, Alibaba Qwen, Tencent Hunyuan, ByteDance Doubao, DeepSeek, Moonshot Kimi, and SiliconFlow — each with a verified base URL, default model, declared capabilities (`chat`, `embeddings`, `images`), and documentation link.

Three safety properties follow from this design:

1. **No SSRF surface.** Because upstream URLs are constants in a registry rather than client-supplied, a caller cannot redirect the server to an

arbitrary internal address.

2. **Explicit key resolution and storage.** Credentials resolve from environment variables first and otherwise from per-user encrypted storage; missing keys produce clear 4xx responses, never silent fallback to a fake model.
3. **Honest capability declarations.** Request/response bodies stream through unchanged, upstream status codes propagate, and unsupported capabilities return 501.

6.4 On-Premises NURBS Expert Chat

In addition to commercial providers, the API exposes an expert chat path (`/ai/chat`) backed by a local model runtime (Ollama) and contextualized with NURBS/Class-A domain information. This gives designers a way to ask "why is this edge failing G1?" or "how do I change the windshield angle without breaking the roof blend?" without sending design data to a third party. Health and model-listing endpoints report whether the local runtime is available.

6.5 A Note on Parameter-Efficient Fine-Tuning

QLoRA-style parameter-efficient fine-tuning (Dettmers et al., 2023) is the natural next step for adapting the expert model to an organization's historical surfacing projects. The current release deliberately does **not** ship a fine-tuning job runner; we treat it as explicitly scoped future work (Section 9) so that the system's claims remain exactly aligned with the executable code.

7. End-to-End Integration in Practice

Two scenarios illustrate how the layers compose.

Scenario A — Proportion optimization. A designer opens the AI designer, loads a sedan preset, and starts a Bayesian session. Each suggestion returns a concrete parameter vector; the backend builds the car, runs the quality evaluator, and the score is observed back into the session. The Three.js viewport updates with the current best, the quality page shows G0/G1/G2 level counts, and the exported sample set feeds the training module. No representation is rebuilt by hand.

Scenario B — Texture- and language-driven delivery. A stylist supplies a lamp texture reference. CLIP scores it on the six style axes; brand

knowledge recommends a matching brand DNA and parameter shifts; the delivery page applies them inside an import→modify→export session and produces STEP output. Along the way the designer consults the local NURBS expert on a continuity warning, and the continuity checker's CSV report documents seam gaps and tangent angles for the engineering hand-off.

In both scenarios the frontend never needs to know whether a capability is local or remote; availability is a runtime property reported by health and capability endpoints, with mock data preventing a black screen when the backend itself is down.

8. Experiments and Evaluation

8.1 Test Baseline

The platform carries **489 automated test cases**, executed independently at each layer:

Layer	Framework	Cases	Scope
algorithm_model	pytest	200	NURBS math, swept/fillet, builders, quality, SOP
backend	pytest	178	routes, schemas, optimizer service, security, persistence
frontend	vitest	111	stores, components, API modules, guards
algorithm self-check	<code>test_all.py</code>	5 modules	one-command end-to-end sanity gate

The self-check builds a parameterized body without mesh post-processing (so section-ring construction order is preserved), extracts a one-side surface grid, and exercises assembly statistics including per-part colors and bounds. During the present review, a contract drift in the assembler's statistics dictionary (callers expected `total_vertices`, which had been removed) and an outdated test fixture (an 8-vertex box reshaped into a section grid) were identified and fixed; all suites then passed. This is reported as evidence of the baseline's role as a genuine regression gate.

8.2 Optimizer Convergence

To validate the acquisition mechanics independent of geometry cost, the optimizer was run on a controlled one-dimensional objective for 15 suggest–observe iterations under both acquisitions. Both EI and UCB drove the incumbent best objective value past the -0.01 band (i.e. effectively to the known optimum) within the 15-round budget, confirming that (i) the

warm-start/fit path is exercised correctly, (ii) acquisition maximization locates informative points rather than repeating the incumbent, and (iii) goal-direction normalization behaves identically for the two acquisitions. The experiment is intentionally small: its purpose is correctness and convergence evidence of the *engine*, not a claim of state-of-the-art sample efficiency against tuned external BO libraries.

8.3 Continuity and Quality Measurements

On the discrete spherical reference surface used by the quality package, the evaluator reports grade D with a G2-level ratio of approximately 0.199 — a transparent baseline showing that raw unfaired geometry does *not* satisfy Class-A curvature requirements and must be improved, which is precisely the condition the optimization/fairing operators are meant to address. The one-side grid extracted from the parameterized body (`n_long=48, n_circ=24` , even ring positions) yields grade C with a G2 ratio of 0.855 but a reflection score of 0.158: the section-based body is already smooth in angular continuity while its curvature-uniformity metric remains below the Class-A band. On these continuous, hard-point-dominated panels the annealing fairer produces negligible change in the graded counts ($\Delta G2 = 0$ over 80 iterations); its measurable effect appears on deliberately perturbed surfaces (e.g. a noisy plane, reflection $0.330 \rightarrow 0.339$), matching the documented engineering guidance that local Class-A fairing should follow hard-point freeze.

For the NURBS body with G1 end caps, the continuity checker applies the 0.1 mm / 1.0° thresholds and reports per-pair positional and angular deviations into TXT and CSV artifacts, giving engineers an immediate seam-by-seam verdict rather than a single opaque score.

8.4 Runtime Performance

Representative durations on the development machine (Windows): the 200-case algorithm test suite runs in roughly 4 seconds; the five-module self-check in roughly 7 seconds; the 178-case backend suite in roughly 28 seconds (including process/session fixtures); the 111-case frontend suite in roughly 1.5 seconds (test duration, excluding environment startup). Suggestion latency is dominated by GP fit and the 4096-point pool prediction, both of which remain comfortably interactive at the current observation counts. Geometry build of the parameterized car and full assembly is on the order of hundreds of milliseconds, which is what makes per-suggestion evaluation in an interactive design loop feasible.

8.5 Failure-Mode Behavior

We additionally verified the degradation contract. With PyTorch missing, training capability endpoints report unavailability and training requests fail explicitly rather than returning fabricated metrics. With no LLM provider key, chat requests return explicit client errors. With CLIP weights unavailable, texture analysis falls back to geometric features and labels the response accordingly. With Redis absent, session storage degrades to in-process memory. The common principle — *signal, never simulate* — is what allows designers to trust the output they do receive.

9. Discussion and Limitations

Parameterization expressiveness. A 22-field explicit vector is interpretable and robust but cannot represent arbitrary stylistic gestures or localized sculpting. Learned or sketch-conditioned control layers (including NURBGen-style generative primitive models) would broaden the shape vocabulary; integrating them while preserving editability is open work.

Surrogate fidelity. The isotropic RBF GP with a random-pool acquisition optimizer is deliberately simple. Anisotropic or learned length scales, non-GP surrogates for high-dimensional spaces, and gradient-aware acquisition optimization would improve scaling beyond 14 dimensions. Multi-fidelity modeling — cheap mesh curvature alongside expensive exact NURBS evaluation — is a natural extension.

Objective definition. Quality scores currently combine curvature statistics and continuity checks; manufacturing constraints, packaging interference, and aerodynamic targets are only partially modeled. Designing objectives that faithfully encode all Class-A acceptance criteria remains the dominant domain challenge, more so than the optimization math itself.

Fine-tuning and provenance. As noted, QLoRA-style fine-tuning on an organization's project history is not yet shipped. When it is, data provenance, versioned corpora, and evaluation against held-out surfacing cases will be required to avoid silently degrading expert advice.

Surrogate/data realism. The classifier's synthetic dataset is deterministic and anchored to real centroids, but it is not a substitute for large labeled corpora of production vehicles. The BO-to-training

bridge is designed precisely so that *real optimization campaigns* progressively replace synthetic samples with observations.

10. Conclusion

EVOLUTION AI demonstrates that the key technologies for modern Class-A surface development — exact NURBS geometry, data-efficient Bayesian optimization, vision-language semantics, and LLMs — can be composed into one coherent, trustworthy web platform rather than remaining isolated tools. The combination of a parameter-driven dual geometry pipeline, a transparent GP+EI/UCB optimizer with a training-data bridge, and a heterogeneous AI fusion layer with truthful degradation shortens the loop from concept proportion to engineering STEP output while keeping every result editable and auditable. The 489-case test baseline, convergence experiments, and continuity measurements provide evidence that the integrated system behaves as specified. Future work will add parameter-efficient expert fine-tuning, richer objectives and constraints, higher-fidelity surrogates, and learned layers that expand the shape vocabulary without sacrificing the exact NURBS guarantee.

References

1. Piegl, L., & Tiller, W. (1997). *The NURBS Book* (2nd ed.). Monographs in Visualization, Springer.
2. Farin, G. (2002). *Curves and Surfaces for CAD: A Practical Guide* (5th ed.). Morgan Kaufmann.
3. Rasmussen, C. E., & Williams, C. K. I. (2006). *Gaussian Processes for Machine Learning*. MIT Press.
4. Jones, D. R., Schonlau, M., & Welch, W. J. (1998). Efficient Global Optimization of Expensive Black-Box Functions. *Journal of Global Optimization*, 13(4), 455–492.
5. Snoek, J., Larochelle, H., & Adams, R. P. (2012). Practical Bayesian Optimization of Machine Learning Algorithms. *Advances in Neural Information Processing Systems (NeurIPS)*, 25.
6. Radford, A., Kim, J. W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., Sastry, G., Askell, A., Mishkin, P., Clark, J., Krueger, G., & Sutskever, I. (2021). Learning Transferable Visual

Models From Natural Language Supervision (CLIP). *Proceedings of ICML*.

7. Dettmers, T., Pagnoni, A., Holtzman, A., & Zettlemoyer, L. (2023).

QLoRA: Efficient Finetuning of Quantized LLMs. *Advances in Neural Information Processing Systems (NeurIPS)*, 36.

8. NURBGen (2026). A generative approach to NURBS geometry. *Proceedings of the AAAI Conference on Artificial Intelligence*, 40.

<https://ojs.aaai.org/index.php/AAAI/article/view/37922>

Appendix A. Mapping Claims to the Source Tree

Paper element	Source location
14-dimension default space, GP, EI/UCB, sessions	<code>backend/app/bayes_optimizer.py</code> ; routes in <code>backend/app/routes/bayes.py</code>
Training feature order, CarTypeMLP, background worker	<code>backend/app/routes/training.py</code>
NURBS basis / curves / surfaces	<code>algorithm_model/freeform/nurbs_core.py</code>
Swept, fillet, free-form surfaces, STEP writer	<code>algorithm_model/freeform/swept_surface.py</code> , <code>fillet_surface.py</code> , <code>step_writer.py</code>
NURBS body and G1 end caps	<code>algorithm_model/car_modeling/body_nurbs.py</code> , <code>body_ends_g1.py</code>
G0/G1 continuity thresholds and reports	<code>algorithm_model/car_modeling/continuity_checker.py</code>
Discrete G0/G1/G2 quality and fairing	<code>algorithm_model/surface_quality/</code>
CLIP six-axis semantics	<code>backend/app/clip_semantics.py</code> ; consumed by <code>routes/texture.py</code>
Brand mapping	<code>backend/app/brand_knowledge.py</code>
Seven-provider LLM proxy	<code>backend/app/routes/llm_proxy.py</code>
Local NURBS expert chat	<code>/ai/chat</code> path and <code>src/api.js</code> (<code>chatWithExpert</code>)
Persistence tables	<code>backend/app/database.py</code>
Test baselines	<code>algorithm_model/tests/</code> , <code>backend/tests/</code> , <code>tests/</code> (vittest), <code>algorithm_model/test_all.py</code>